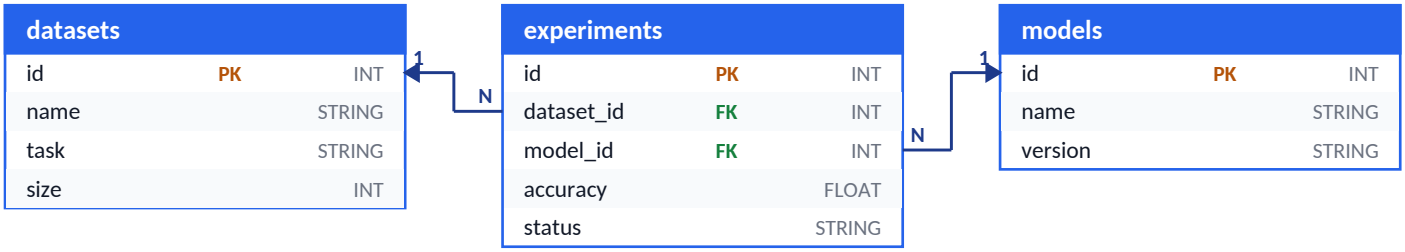
Отчёт по лабораторной работе №1

MiniDB — собственная файловая мини-СУБД на Python (без SQLite, ORM, pandas и JSON-хранилищ)

1. Схема базы данных

Три таблицы и две связи «один ко многим»: один датасет и одна модель могут участвовать во многих экспериментах. PK — первичный ключ, FK — внешний. Движок не даст добавить эксперимент с несуществующим датасетом или моделью и не даст удалить датасет или модель, на которые ссылаются эксперименты.



Типы данных: INT и FLOAT занимают по 8 байт, STRING — 64 байта. Схема для dbdiagram.io лежит в файле schema.dbml.

2. Формат хранения

Каждая таблица состоит из двух файлов. **<имя>.meta** — схема обычным текстом (по строке на столбец: имя, тип, пометки pk или fk=таблица.поле). **<имя>.dat** — бинарные записи **фиксированной длины**: 1 байт флага «жива/удалена» и поля подряд (модуль struct). Запись номер k лежит со смещения k × размер записи, поэтому её можно найти и перезаписать на месте. Раскладка записи таблицы experiments (97 байт, не в масштабе):



Таблица	Размер записи	Файл .dat после seed
datasets	145 байт	5 × 145 = 725 байт
models	137 байт	5 × 137 = 685 байт
experiments	97 байт	20 × 97 = 1940 байт

INSERT — запись дописывается в конец файла.

SELECT — файл читается по записям (Seq Scan) либо запись находится через индекс.

UPDATE — запись перезаписывается на месте (seek и write).

DELETE — в поле флага пишется 0, запись остаётся в файле, но пропускается при чтении.

Данные переживают перезапуск: всё состояние хранится только в файлах.

3. Примеры операций (команды и вывод программы)

```
select experiments id,accuracy where accuracy >= 0.9 and status = completed
id | accuracy
1 | 0.93
3 | 0.91
5 | 0.9
7 | 0.95
12 | 0.97
13 | 0.96
(6 строк)
план: Seq Scan
insert experiments dataset_id=1 model_id=2 accuracy=0.5 status=running
добавлено: {'dataset_id': 1, 'model_id': 2, 'accuracy': 0.5, 'status': 'running', 'id': 21}
select experiments where id = 21
id | dataset_id | model_id | accuracy | status
21 | 1 | 2 | 0.5 | completed
(1 строк)
план: Index Scan по id
update experiments set status=completed where id = 21
изменено: 1
delete experiments where id = 21
удалено: 1
```

```
insert experiments dataset_id=99 model_id=1 accuracy=0.5 status=x
Ошибка: нарушение FK: в datasets нет записи с id=99
delete datasets where id = 1
Ошибка: нельзя удалить: на запись ссылается experiments.dataset_id
select experiments join datasets on experiments.dataset_id = datasets.id join models on experiments.model_id = models.id experiments.id,datasets.name,models.name,experiments.accuracy where experiments.id <= 3
experiments.id | datasets.name | models.name | experiments.accuracy
1 | CIFAR-10 | BERT | 0.93
2 | IMDB | LogReg | 0.79
3 | Iris | CNN | 0.91
(3 строк)
select experiments where status = failed or accuracy < 0.7
id | dataset_id | model_id | accuracy | status
9 | 4 | 2 | 0.98 | failed
10 | 1 | 5 | 0.69 | running
11 | 4 | 5 | 0.69 | completed
...
```

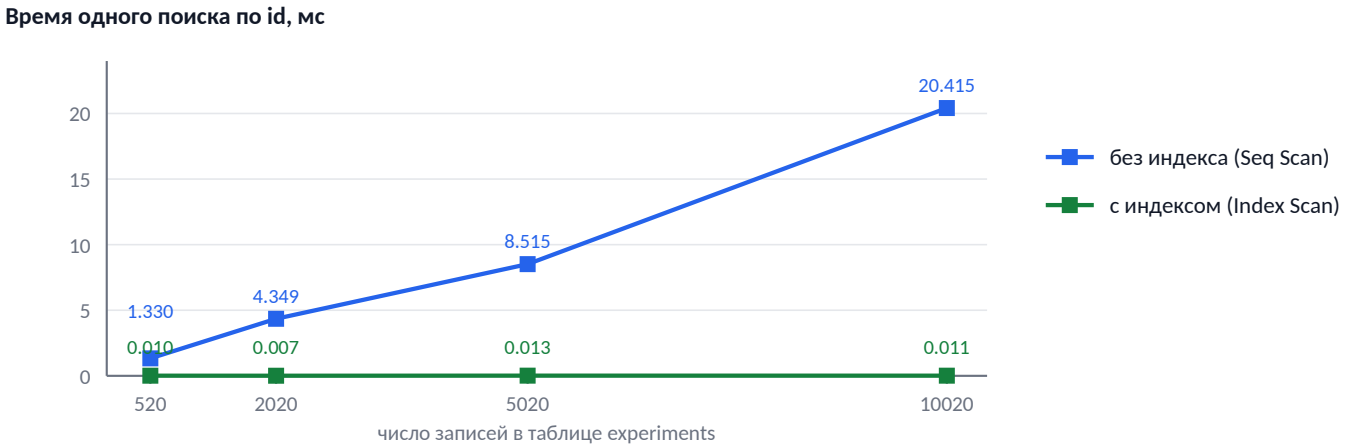
Реализованы: CREATE TABLE, INSERT, SELECT, UPDATE, DELETE, выбор столбцов, условия = != > < >= <= с and/or, авто-ID, запрет дублей PK, проверка типов и внешних ключей, INNER JOIN, индекс, backup/restore, интерфейс командной строки.

4. Индекс id -> offset и сравнение поиска до и после индекса

Индекс — словарь «значение первичного ключа -> смещение записи в .dat». Он строится одним проходом по файлу при первом поиске по id, а затем обновляется при INSERT и DELETE. Поиск по id становится одним прыжком (seek) прямо к записи вместо чтения всего файла. После каждого select программа печатает план выполнения: «Index Scan по id» или «Seq Scan» (при фильтре по другим полям индекс не используется).

Записей в experiments	Без индекса, мс на поиск	С индексом, мс на поиск	Построение индекса, мс	Ускорение
520	1.330	0.010	0.88	x128
2020	4.349	0.007	2.02	x654
5020	8.515	0.013	7.57	x660
10020	20.415	0.011	13.74	x1817

Замер: 300 поисков по случайным id в таблице experiments, данные созданы командами seed и gen. Время поиска без индекса растёт пропорционально числу записей (полный просмотр файла, O(n)), с индексом остаётся почти постоянным (O(1)). Индекс строится один раз и окупается уже после нескольких поисков. Корректность проверена: на 400 случайных запросах результаты с индексом и без него совпали.



Выводы по замеру

• Без индекса время поиска растёт вместе с таблицей: при росте числа записей примерно в 19 раз поиск замедлился примерно в 15 раз, потому что каждый поиск читает весь файл.

• С индексом поиск по первичному ключу занимает около 0,01 мс и почти не зависит от размера таблицы: нужно одно обращение к словарю и одно чтение записи.

• У индекса есть цена: память под словарь, один проход по файлу при построении и обновление при каждом INSERT и DELETE. Поэтому индекс сделан только по первичному ключу и используется для условия id = значение; остальные фильтры (например, по status) выполняются через Seq Scan.

5. Backup/restore и воспроизводимость

backup файл.zip архивирует файлы базы, **restore файл.zip** заменяет текущие данные содержимым архива (архив проверяется до удаления старых файлов). Проверено: после удаления 10 записей и restore все записи вернулись. **Запуск:** python main.py, затем seed, gen 5000, bench 300. Команды seed и gen используют фиксированный random.seed, поэтому данные одинаковы при каждом запуске. **Ограничения:** нет транзакций и журнала, индекс только по первичному ключу, STRING не длиннее 64 байт.